МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ
Ордена Трудового Красного Знамени федеральное государственное
Бюджетное образовательное учреждение высшего образования
МОСКОВСКИЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ СВЯЗИ И ИНФОРМАТИКИ

Лабораторная работа № 4

по дисциплине «Математические основы баз данных»

по теме: «Оконные функции и аналитика данных»

Выполнил студент группы:
БВТ2505
Соронина Арина Андреевна
Проверил: Быватов Семён Юрьевич

Москва 2026

Цель и задачи работы:

Цель: реализация набора аналитических запросов с использованием оконных функций.

Задачи:

1. Постройте рейтинг внутри каждой категории с помощью ROW_NUMBER, RANK и DENSE_RANK; поясните, чем различаются их результаты на строках с равными значениями.

2. Для каждой категории выведите три самых дорогих товара (нумерация в окне и фильтрация через CTE).

3. Вычислите нарастающий итог выручки по дням оконной агрегатной функцией.

4. С помощью SUM(...) OVER (PARTITION BY ...) посчитайте долю выручки каждого продавца в выручке его категории.

5. Примените LAG и LEAD для сравнения выручки текущего дня с предыдущим (вычислите прирост).

6. Решите одну из задач двумя способами — оконной функцией и обычной группировкой — и поясните, почему оконная функция здесь удобнее.

Выполнение

1. *Рейтинг по продолжительности внутри каждого жанра через ROW_NUMBER, RANK и DENSE_RANK.*

```
SELECT
    g.name AS genre_name,
    m.title AS movie_title,
    m.duration_min,
    ROW_NUMBER() OVER w AS row_num,
    RANK() OVER w AS rnk,
    DENSE_RANK() OVER w AS dense_rnk
FROM movies m
JOIN genres g ON m.genre_id = g.genre_id
WINDOW w AS (PARTITION BY m.genre_id ORDER BY m.duration_min DESC);
```

	genre_name character varying (100)	movie_title character varying (200)	duration_min integer	row_num bigint	rnk bigint	dense_rnk bigint
1	Драма	Ночь потерянных душ	179	1	1	1
2	Драма	Побег потерянных душ 4	179	2	1	1
3	Драма	Легенда большого города 2	179	3	1	1
4	Драма	Тайна ночного патруля	176	4	4	2
5	Драма	Город чужих берегов 4	176	5	4	2

Различия между функциями на строках с одинаковыми значениями:

- ROW_NUMBER всегда присваивает строго уникальные последовательные номера (1, 2, 3, 4), не учитывая совпадение длительности.
- RANK присваивает одинаковый ранг фильмам с одинаковой длительностью, но образует пропуски в нумерации на последующих строках (например: 1, 1, 3, 4).
- DENSE_RANK также присваивает равным строкам одинаковый ранг, но не оставляет пропусков в ряду чисел (например: 1, 1, 2, 3).

2. *Топ-3 самых продолжительных фильма в каждом жанре (CTE + оконная фильтрация).*

```
WITH RankedMovies AS (
    SELECT
        g.name AS genre_name,
        m.title AS movie_title,
        m.duration_min,
        ROW_NUMBER() OVER (
            PARTITION BY m.genre_id
            ORDER BY m.duration_min DESC
        ) AS rn
    FROM movies m
    JOIN genres g ON m.genre_id = g.genre_id
)
SELECT
    genre_name,
    movie_title,
    duration_min,
    rn AS position
FROM RankedMovies
WHERE rn <= 3
ORDER BY genre_name, rn;
```

	genre_name character varying (100)	movie_title character varying (200)	duration_min integer	position bigint
1	Боевик	Побег вечной зимы	178	1
2	Боевик	Возвращение потерянных душ 4	177	2
3	Боевик	Легенда старого замка	177	3
4	Драма	Ночь потерянных душ	179	1
5	Драма	Побег потерянных душ 4	179	2

3. Нарастающий итог выручки по дням оконной агрегатной функцией

```
SELECT
    paid_at::date AS payment_date,
    SUM(amount) AS daily_revenue,
    SUM(SUM(amount)) OVER (ORDER BY paid_at::date) AS running_total
FROM payments
GROUP BY paid_at::date
ORDER BY payment_date;
```

	payment_date date	daily_revenue numeric	running_total numeric
1	2026-08-01	34250.00	34250.00
2	2026-08-02	24500.00	58750.00
3	2026-08-03	40750.00	99500.00
4	2026-08-04	64650.00	164150.00
5	2026-08-05	40600.00	204750.00

4. Доля выручки каждого прокатчика в выручке жанра (SUM OVER PARTITION BY).

```
WITH DistributorGenreRevenue AS (
    SELECT
        g.genre_id,
        g.name AS genre_name,
        d.company_name AS distributor_name,
        SUM(t.price) AS distributor_revenue
    FROM tickets t
    JOIN sessions s ON t.session_id = s.session_id
    JOIN movies m ON s.movie_id = m.movie_id
    JOIN genres g ON m.genre_id = g.genre_id
    JOIN distributors d ON m.distributor_id = d.distributor_id
    GROUP BY g.genre_id, g.name, d.company_name
)
SELECT
    genre_name,
    distributor_name,
    distributor_revenue,
    SUM(distributor_revenue) OVER (PARTITION BY genre_id) AS total_genre_revenue,
    ROUND(
        distributor_revenue * 100.0 / SUM(distributor_revenue) OVER (PARTITION BY
genre_id),
        2
    ) AS revenue_share_pct
FROM DistributorGenreRevenue
ORDER BY genre_name, distributor_revenue DESC;
```

	genre_name character varying (100)	distributor_name character varying (150)	distributor_revenue numeric	total_genre_revenue numeric	revenue_share_pct numeric
1	Боевик	Universal Pictures	34700.00	274550.00	12.64
2	Боевик	Централ Партнершип	33250.00	274550.00	12.11
3	Боевик	Sony Pictures	30850.00	274550.00	11.24
4	Боевик	Warner Bros.	29800.00	274550.00	10.85
5	Боевик	Paramount Pictures	28550.00	274550.00	10.40

5. *Сравнение выручки текущего дня с предыдущим через LAG и LEAD (вычисление прироста).*

```
WITH DailySales AS (
    SELECT
        paid_at::date AS day,
        SUM(amount) AS day_revenue
    FROM payments
    GROUP BY paid_at::date
)
SELECT
    day,
    day_revenue,
    LAG(day_revenue) OVER (ORDER BY day) AS prev_day_revenue,
    LEAD(day_revenue) OVER (ORDER BY day) AS next_day_revenue,
    day_revenue - LAG(day_revenue) OVER (ORDER BY day) AS daily_growth
FROM DailySales
ORDER BY day;
```

	day date	day_revenue numeric	prev_day_revenue numeric	next_day_revenue numeric	daily_growth numeric
1	2026-08-...	34250.00	[null]	24500.00	[null]
2	2026-08-...	24500.00	34250.00	40750.00	-9750.00
3	2026-08-...	40750.00	24500.00	64650.00	16250.00
4	2026-08-...	64650.00	40750.00	40600.00	23900.00
5	2026-08-...	40600.00	64650.00	45250.00	-24050.00

6. *Решение задачи двумя способами: сопоставление фильма со средней продолжительностью его жанра.*

Способ 1: с помощью оконной функции.

```
SELECT

    m.title,

    m.duration_min,

    m.genre_id,

    ROUND(AVG(m.duration_min)    OVER    (PARTITION    BY    m.genre_id),    1)    AS
avg_genre_duration,

    ROUND(m.duration_min - AVG(m.duration_min) OVER (PARTITION BY m.genre_id), 1) AS
diff_from_avg

FROM movies m;
```

	title character varying (200)	duration_min integer	genre_id integer	avg_genre_duration numeric	diff_from_avg numeric
1	Тень старого замка 3	171	1	131.9	39.1
2	Побег третьей планеты 4	85	1	131.9	-46.9
3	Хозяин последнего героя 3	85	1	131.9	-46.9
4	Путь старого замка	124	1	131.9	-7.9
5	Битва чужих берегов 2	129	1	131.9	-2.9

Способ 2: с помощью классической группировки (GROUP BY).

```
SELECT

    m.title,

    m.duration_min,

    m.genre_id,

    ROUND(ga.avg_genre_duration, 1) AS avg_genre_duration,

    ROUND(m.duration_min - ga.avg_genre_duration, 1) AS diff_from_avg

FROM movies m

JOIN (

    SELECT

        genre_id,

        AVG(duration_min) AS avg_genre_duration

    FROM movies

    GROUP BY genre_id

) ga ON m.genre_id = ga.genre_id;
```

	title character varying (200)	duration_min integer	genre_id integer	avg_genre_duration numeric	diff_from_avg numeric
1	Битва потерянных душ 4	135	7	126.2	8.8
2	Хозяин третьей планеты 4	158	5	132.1	25.9
3	Тайна третьей планеты 4	83	4	131.0	-48.0
4	Зов ночного патруля 3	118	6	132.7	-14.7
5	Зов третьей планеты 2	100	7	126.2	-26.2

Обоснование: GROUP BY объединяет (схлопывает) строки группы в одну, из-за чего исходные записи о фильмах теряются. Чтобы вывести детальные поля фильма рядом со средним значением по жанру, при использовании GROUP BY приходится писать подзапрос и делать дополнительное соединение JOIN. Оконная функция не схлопывает строки, сохраняя каждую детальную запись в результате и вычисляя сводный показатель параллельно за один проход.

Вывод

В ходе выполнения работы были освоены и применены на практике ключевые инструменты SQL в СУБД PostgreSQL. Были построены многотабличные запросы с использованием `INNER JOIN` и `LEFT JOIN` для объединения данных и выявления отсутствующих связей, а также реализована аналитическая группировка с фильтрацией агрегатов через `GROUP BY` и `HAVING`. Для комплексного отбора данных задействованы скалярные и предикатные подзапросы с `IN` и `EXISTS`. С помощью обобщенных табличных выражений (`CTE`), включая рекурсивный запрос, структурированы вычисления и сгенерирована календарная сетка сеансов, а созданные представления (`VIEW`) инкапсулировали сложную логику соединений в готовые типовые отчёты. Все требования технического задания выполнены в полном объёме.